

## DAA

### LINEAR SEARCH

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n1 = sc.nextInt();
        int[] inventory = new int[n1];
        for (int i = 0; i < n1; i++) {
            inventory[i] = sc.nextInt();
        }
        int n2 = sc.nextInt();
        int[] checkList = new int[n2];
        for (int i = 0; i < n2; i++) {
            checkList[i] = sc.nextInt();
        }
        for (int id : checkList) {
            int position = -1;
            for (int i = 0; i < n1; i++) {
                if (inventory[i] == id) {
                    position = i + 1;
                    break;
                }
            }
            if (position != -1) {
                System.out.println(id + " found at position " + position);
            } else {
                System.out.println(id + " not found in the inventory");
            }
        }
        sc.close();
    }
}
```

### BINARY SEARCH

```
import java.util.*;
```

```
public class Main {  
    public static int binarySearch(int[] arr, int target) {  
        int low = 0;  
        int high = arr.length - 1;  
        while (low <= high) {  
            int mid = (low + high) / 2;  
            if (arr[mid] == target) {  
                return mid + 1;  
            } else if (arr[mid] < target) {  
                low = mid + 1;  
            } else {  
                high = mid - 1;  
            }  
        }  
        return -1;  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n1 = sc.nextInt();  
        int[] inventory = new int[n1];  
        for (int i = 0; i < n1; i++) {  
            inventory[i] = sc.nextInt();  
        }  
        Arrays.sort(inventory);  
        int n2 = sc.nextInt();  
        int[] checkList = new int[n2];  
        for (int i = 0; i < n2; i++) {  
            checkList[i] = sc.nextInt();  
        }  
        for (int id : checkList) {  
            int position = binarySearch(inventory, id);  
            if (position != -1) {  
                System.out.println(id + " found at position " + position);  
            } else {  
                System.out.println(id + " not found in the inventory");  
            }  
        }  
    }  
}
```

```
        sc.close();
    }
}
```

Input 1 :

```
5
10 20 30 40 50
4
20 30 40 60
```

Output 1 :

```
20 found at position 2
30 found at position 3
40 found at position 4
60 not found in the inventory
```

## ADJACENCY LIST

```
import java.util.*;
```

```
class Graph {
    int vertices;
    List<List<Integer>> adjacencyList;

    Graph(int vertices) {
        this.vertices = vertices;
        adjacencyList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        adjacencyList.get(u).add(v);
        adjacencyList.get(v).add(u);
    }

    public void display() {
        for (int i = 0; i < vertices; i++) {
            System.out.print("Vertex " + i + ": ");
        }
    }
}
```

```

        Collections.sort(adjacencyList.get(i));
        for (int neighbor : adjacencyList.get(i)) {
            System.out.print(neighbor + " -> ");
        }
        System.out.println();
    }
}
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int vertices = sc.nextInt();
        int edges = sc.nextInt();
        Graph g = new Graph(vertices);
        for (int i = 0; i < edges; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            g.addEdge(u, v);
        }
        g.display();
        sc.close();
    }
}

```

## INSERTION SORT

```

import java.util.*;
class Solution{
    public static void main(String[]args){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int[]arr=new int[n];
        for(int i=0;i<n;i++){
            arr[i]=sc.nextInt();
        }
        insertion_sort(arr);
        for(int j=0;j<n;j++){
            System.out.print(arr[j]+" ");
        }
    }
}

```

```

    }
    public static void insertion_sort(int[] arr){
        for(int i=1;i<arr.length;i++){
            int temp=arr[i];
            int j=i-1;
            while(j>=0 && arr[j]>temp){
                arr[j+1]=arr[j];
                j--;
            }
            arr[j+1]=temp;
        }
    }
}

```

## MERGE SORT

```

import java.util.*;

public class Main {
    public static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++) {
            L[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) {
            R[j] = arr[mid + 1 + j];
        }

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
}

```

```

        j++;
    }
    k++;
}

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] arr = new int[n];

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    mergeSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }
}

```

```

    }

    sc.close();
}
}

```

## ODD-EVEN BASED INSERTION SORT

```

import java.util.*;

public class Main {
    public static void insertionSortAsc(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    public static void insertionSortDesc(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] < key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {

```

```

        arr[i] = sc.nextInt();
    }

    int oddCount = (n + 1) / 2;
    int evenCount = n / 2;
    int[] odd = new int[oddCount];
    int[] even = new int[evenCount];

    int oi = 0, ei = 0;
    for (int i = 0; i < n; i++) {
        if ((i + 1) % 2 == 0) {
            even[ei++] = arr[i];
        } else {
            odd[oi++] = arr[i];
        }
    }

    insertionSortDesc(odd);
    insertionSortAsc(even);

    oi = 0;
    ei = 0;
    for (int i = 0; i < n; i++) {
        if ((i + 1) % 2 == 0) {
            arr[i] = even[ei++];
        } else {
            arr[i] = odd[oi++];
        }
    }

    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }

    sc.close();
}
}

```

## **K closest prices to a target value**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int target = sc.nextInt();
        int k = sc.nextInt();

        boolean[] used = new boolean[n];
        for (int i = 0; i < k; i++) {
            int minDiff = Integer.MAX_VALUE;
            int index = -1;
            for (int j = 0; j < n; j++) {
                int diff = Math.abs(arr[j] - target);
                if (!used[j]) {
                    if (diff < minDiff || (diff == minDiff && arr[j] < arr[index])) {
                        minDiff = diff;
                        index = j;
                    }
                }
            }
            used[index] = true;
            System.out.print(arr[index] + " ");
        }
        sc.close();
    }
}
```

## **BUBBLE SORT**

```
import java.util.*;

class Main{
    public static void main(String[]args){
```

```

Scanner sc=new Scanner(System.in);
int n=sc.nextInt();
int[]arr=new int[n];
for(int i=0;i<n;i++){
    arr[i]=sc.nextInt();
}
bubbleSort(arr);
for(int i=0;i<n;i++){
    System.out.print(arr[i]);
}
}
public static void bubbleSort(int[]arr){
    int n=arr.length;
    for(int i=0;i<n;i++){
        for(int j=0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]){
                int temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
}
}

```

## SELECTION SORT

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;

```

```

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }

    sc.close();
}

```

## QUICK SORT

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        quickSort(arr, 0, n - 1);

        System.out.print("Final Sorted Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }

        sc.close();
    }
}

```

```

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        System.out.print("After partition with pivot " + arr[p] + ": ");
        printArray(arr);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

public static void printArray(int[] arr) {
    for (int x : arr) {
        System.out.print(x + " ");
    }
    System.out.println();
}
}

```

**Input:**

6  
10 7 8 9 1 5

## Output:

After partition with pivot 5: 1 5 8 9 10 7  
After partition with pivot 7: 1 5 7 9 10 8  
After partition with pivot 8: 1 5 7 8 10 9  
After partition with pivot 9: 1 5 7 8 9 10  
Final Sorted Array: 1 5 7 8 9 10

## LINEAR PROBING

```
import java.util.*;

public class LinearProbing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter hash table size: ");
        int size = sc.nextInt();
        int[] table = new int[size];
        Arrays.fill(table, -1);

        System.out.print("Enter number of keys: ");
        int n = sc.nextInt();
        int[] keys = new int[n];
        System.out.println("Enter keys:");
        for (int i = 0; i < n; i++) {
            keys[i] = sc.nextInt();
        }

        for (int key : keys) {
            int index = key % size;
            while (table[index] != -1) {
                index = (index + 1) % size;
            }
            table[index] = key;
        }

        System.out.println("\nHash Table (Linear Probing):");
        for (int i = 0; i < size; i++) {
            System.out.println(i + " --> " + table[i]);
        }
    }
}
```

```
}  
}
```

## QUADRATIC PROBING

```
import java.util.*;  
  
public class QuadraticProbing {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter hash table size: ");  
        int size = sc.nextInt();  
        int[] table = new int[size];  
        Arrays.fill(table, -1);  
  
        System.out.print("Enter number of keys: ");  
        int n = sc.nextInt();  
        int[] keys = new int[n];  
        System.out.println("Enter keys:");  
        for (int i = 0; i < n; i++) {  
            keys[i] = sc.nextInt();  
        }  
  
        for (int key : keys) {  
            int index = key % size;  
            int i = 0;  
            while (table[(index + i * i) % size] != -1) {  
                i++;  
            }  
            table[(index + i * i) % size] = key;  
        }  
  
        System.out.println("\nHash Table (Quadratic Probing):");  
        for (int i = 0; i < size; i++) {  
            System.out.println(i + " --> " + table[i]);  
        }  
    }  
}
```

## DOUBLE HASHING

```

import java.util.*;
public class DoubleHashing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter hash table size: ");
        int size = sc.nextInt();
        int[] table = new int[size];
        Arrays.fill(table, -1);

        System.out.print("Enter number of keys: ");
        int n = sc.nextInt();
        int[] keys = new int[n];
        System.out.println("Enter keys:");
        for (int i = 0; i < n; i++) {
            keys[i] = sc.nextInt();
        }

        for (int key : keys) {
            int index = hash1(key, size);
            int step = hash2(key);
            while (table[index] != -1) {
                index = (index + step) % size;
            }
            table[index] = key;
        }

        System.out.println("\nHash Table (Double Hashing):");
        for (int i = 0; i < size; i++) {
            System.out.println(i + " --> " + table[i]);
        }
    }

    static int hash1(int key, int size) {
        return key % size;
    }

    static int hash2(int key) {
        return 7 - (key % 7); // secondary hash function
    }
}

```

## **PRIORITY QUEUE**

### **i) CUSTOM COMPARATOR**

```
import java.util.*;

class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Student> pq = new PriorityQueue<>((a, b) -> b.marks - a.marks);

        pq.add(new Student("Asha", 85));
        pq.add(new Student("Bala", 92));
        pq.add(new Student("Chinmay", 78));

        System.out.println("Students in order of marks (highest first):");
        while (!pq.isEmpty()) {
            Student s = pq.poll();
            System.out.println(s.name + " - " + s.marks);
        }
    }
}
```

### **ii) MAX HEAP**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());

        pq.add(40);
```

```

pq.add(10);
pq.add(30);
pq.add(20);

System.out.println("Priority Queue (Max-Heap): " + pq);
System.out.println("Top element (peek): " + pq.peek());

System.out.println("Elements in descending order:");
while (!pq.isEmpty()) {
    System.out.print(pq.poll() + " ");
}
}
}

```

### iii) MIN HEAP

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(40);
        pq.add(10);
        pq.add(30);
        pq.add(20);

        System.out.println("Priority Queue: " + pq);
        System.out.println("Top element (peek): " + pq.peek());

        System.out.println("Elements in sorted order:");
        while (!pq.isEmpty()) {
            System.out.print(pq.poll() + " ");
        }
    }
}

```

## QuickSort + Merge Intervals

```
import java.util.*;

public class MergeIntervals {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] intervals = new int[n][2];
        for (int i = 0; i < n; i++) {
            intervals[i][0] = sc.nextInt();
            intervals[i][1] = sc.nextInt();
        }

        quickSort(intervals, 0, n - 1);

        List<int[]> merged = new ArrayList<>();
        int[] current = intervals[0];
        merged.add(current);

        for (int[] interval : intervals) {
            if (interval[0] <= current[1]) {
                current[1] = Math.max(current[1], interval[1]);
            } else {
                current = interval;
                merged.add(current);
            }
        }

        System.out.println("Merged intervals:");
        for (int[] m : merged) {
            System.out.println "[" + m[0] + ", " + m[1] + "]");
        }
    }

    public static void quickSort(int[][] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }
}
```

```

    }
}

public static int partition(int[][] arr, int low, int high) {
    int pivot = arr[high][0];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j][0] <= pivot) {
            i++;
            int[] temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int[] temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
}

```

## CLOSEST PAIR OF POINTS

```

import java.util.*;
class Pairs{
    static class Point{
        int x;
        int y;
        Point(int x,int y){
            this.x=x;
            this.y=y;
        }
    }
    public static double dist(Point p1,Point p2){
        return Math.sqrt((p2.x-p1.x)*(p2.x-p1.x)+(p2.y-p1.y)*(p2.y-p1.y));
    }
    public static void main(String[]args){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        Point[]p=new Point[n];
    }
}

```

```

for(int i=0;i<n;i++){
    p[i]=new Point(sc.nextInt(),sc.nextInt());
}
double m=100.00;
Point[]pair=new Point[2];
for(int i=0;i<n;i++){
    for(int j=i+1;j<n;j++){
        double d=dist(p[i],p[j]);
        if(d<m){
            m=d;
            pair[0]=p[i];
            pair[1]=p[j];
        }
    }
}
System.out.print("Closest pair of points: "+(" "+pair[0].x+", "
+pair[0].y+") and (" +pair[1].x+", "+ pair[1].y +")" );
System.out.println();
System.out.print("Distance: ");
System.out.print(String.format("%.2f",m));

}
}

```

## **BFS TRAVERSAL**

```

import java.util.*;

public class BFS_Traversal {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int vertices = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int edges = sc.nextInt();

        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {

```

```

        adj.add(new ArrayList<>());
    }

    System.out.println("Enter edges (u v):");
    for (int i = 0; i < edges; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        adj.get(u).add(v);
        adj.get(v).add(u); // Remove this line if graph is directed
    }

    System.out.print("Enter starting node: ");
    int start = sc.nextInt();

    bfs(adj, vertices, start);
}

public static void bfs(List<List<Integer>> adj, int vertices, int start) {
    boolean[] visited = new boolean[vertices];
    Queue<Integer> q = new LinkedList<>();

    visited[start] = true;
    q.add(start);

    System.out.print("BFS Traversal: ");
    while (!q.isEmpty()) {
        int node = q.poll();
        System.out.print(node + " ");
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.add(neighbor);
            }
        }
    }
}
}

```

## DFS TRAVERSAL

```

import java.util.*;

public class DFS_Traversal {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int vertices = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int edges = sc.nextInt();

        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adj.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < edges; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            adj.get(u).add(v);
            adj.get(v).add(u); // Remove this line for directed graph
        }

        System.out.print("Enter starting node: ");
        int start = sc.nextInt();

        boolean[] visited = new boolean[vertices];
        System.out.print("DFS Traversal: ");
        dfs(adj, visited, start);
    }

    public static void dfs(List<List<Integer>> adj, boolean[] visited, int node) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                dfs(adj, visited, neighbor);
            }
        }
    }
}

```

```

    }
}
}
}

```

## KRUSKAL'S ALGORITHM

```

import java.util.*;

class Edge implements Comparable<Edge> {
    int src, dest, cost;

    Edge(int src, int dest, int cost) {
        this.src = src;
        this.dest = dest;
        this.cost = cost;
    }

    public int compareTo(Edge other) {
        return this.cost - other.cost;
    }
}

public class KruskalMST {
    static int find(int parent[], int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent, parent[i]);
    }

    static void union(int parent[], int rank[], int x, int y) {
        int rootX = find(parent, x);
        int rootY = find(parent, y);
        if (rank[rootX] < rank[rootY])
            parent[rootX] = rootY;
        else if (rank[rootX] > rank[rootY])
            parent[rootY] = rootX;
        else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
    }
}

```

```
}  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    int N = sc.nextInt();  
    int R = sc.nextInt();  
  
    List<Edge> edges = new ArrayList<>();  
    for (int i = 0; i < R; i++) {  
        int u = sc.nextInt();  
        int v = sc.nextInt();  
        int cost = sc.nextInt();  
        edges.add(new Edge(u, v, cost));  
    }  
  
    Collections.sort(edges);  
  
    int[] parent = new int[N];  
    int[] rank = new int[N];  
    for (int i = 0; i < N; i++) {  
        parent[i] = i;  
        rank[i] = 0;  
    }  
  
    List<Edge> result = new ArrayList<>();  
    int totalCost = 0;  
  
    for (Edge e : edges) {  
        int rootU = find(parent, e.src);  
        int rootV = find(parent, e.dest);  
  
        if (rootU != rootV) {  
            result.add(e);  
            totalCost += e.cost;  
            union(parent, rank, rootU, rootV);  
        }  
    }  
}
```

```

        System.out.println("Minimum Spanning Tree:");
        for (Edge e : result) {
            System.out.println("Building " + e.src + " - Building " + e.dest + ": " + e.cost);
        }

        System.out.println("Total Cost for Installation: " + totalCost);
    }
}

```

## PRIMS'S ALGORITHM

```

import java.util.*;

public class PrimMST {
    static int findMinKey(int key[], boolean mstSet[], int n) {
        int min = Integer.MAX_VALUE, minIndex = -1;
        for (int v = 0; v < n; v++) {
            if (!mstSet[v] && key[v] < min) {
                min = key[v];
                minIndex = v;
            }
        }
        return minIndex;
    }

    static void primMST(int graph[][], int n) {
        int parent[] = new int[n];
        int key[] = new int[n];
        boolean mstSet[] = new boolean[n];

        for (int i = 0; i < n; i++) {
            key[i] = Integer.MAX_VALUE;
            mstSet[i] = false;
        }

        key[0] = 0;
        parent[0] = -1;

        for (int count = 0; count < n - 1; count++) {
            int u = findMinKey(key, mstSet, n);

```

```

        mstSet[u] = true;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
}

int totalCost = 0;
System.out.println("Minimum Spanning Tree:");
for (int i = 1; i < n; i++) {
    System.out.println("Building " + parent[i] + " - Building " + i + ": " +
graph[i][parent[i]]);
    totalCost += graph[i][parent[i]];
}
System.out.println("Total Cost for Installation: " + totalCost);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int r = sc.nextInt();

    int graph[][] = new int[n][n];
    for (int i = 0; i < r; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        int cost = sc.nextInt();
        graph[u][v] = cost;
        graph[v][u] = cost; // undirected graph
    }

    primMST(graph, n);
}
}

```